



UNIVERSIDAD DEL VALLE
FACULTAD DE INGENIERÍA

Desarrollo del Segundo Examen Parcial de Sistemas Operativos

SISTEMAS OPERATIVOS

Docente: Jefferson Amado Peña Torres

David Taborda Montenegro
202242264 – 3743
Ingeniería de Sistemas

Santiago de Cali

Julio de 2026

1. Introducción.

Para el desarrollo de esta segunda actividad evaluativa, también de carácter práctico, se ha planteado la necesidad de analizar, diseñar e implementar uno de los componentes más críticos e indispensables en la arquitectura de los sistemas operativos modernos, complementario al planificador: el sistema de Administración de Memoria. Dicho componente del kernel, es el encargado de manejar la memoria RAM física, y dado que representa un recurso finito y compartido de forma concurrente por múltiples procesos, debe ser capaz de identificar, determinar y alojar cada proceso que nace en el equipo de cómputo, y permitirle coexistir con otros, compartiendo la memoria principal. Además, su buen planteamiento permitirá que la gestión ineficiente de este recurso no restrinja la capacidad de multiprogramación del sistema, ni que degrade directamente el rendimiento global del hardware, ya que evitaría la ocurrencia de fenómenos de fragmentación y desaprovechamiento del espacio direccionable.

Es así que, con el objetivo de abordar la propuesta hecha por el enunciado del examen parcial, este trabajo se fundamenta en la implementación rigurosa de dos grandes enfoques de gestión de memoria a través de un simulador web interactivo. El primer enfoque aborda el concepto de **Asignación Contigua de Memoria** con particiones, utilizando tres algoritmos de asignación presentados en el curso: *First Fit* (Primer Ajuste), *Best Fit* (Mejor Ajuste) y *Worst Fit* (Peor Ajuste), exponiendo matemáticamente sus impactos en la fragmentación interna y externa del sistema. Del mismo modo, el segundo enfoque se adentra en la sección de **Memoria Virtual por Paginación de un solo nivel**, recreando de manera exacta el comportamiento en hardware de la Unidad de Gestión de Memoria (MMU) para la descomposición y traducción de direcciones lógicas a físicas, así como el control de excepciones de conocidas como *Page Faults* (Fallos de Página).

Para tal finalidad y para desarrollar un buen trabajo de código, toda la arquitectura del simulador se encuentra soportada bajo el paradigma de Programación Orientada a Objetos (POO) utilizando el lenguaje Python y su micro-framework Flask, encargado de direccionar las peticiones hechas desde la Interfaz Gráfica de Usuario (GUI) para el backend, generando el control de las reglas lógicas y estructuras de los simuladores en abstracciones modulares, seguras y altamente escalables. Adicionalmente, el ecosistema de la aplicación ha sido implementado bajo la infraestructura de contenedores mediante la herramienta Docker, garantizando su aislamiento, portabilidad y despliegue inmediato en todo tipo de entorno.

2. Implementaciones de los Simuladores.

2.1. Implementación de la Asignación de Memoria.

Dado que los simuladores solicitados buscan representar estos comportamientos de gestión de memoria y de manejo de los datos dentro de ella, y que se están aplicando tres algoritmos de asignación, se plantearon dos implementaciones para cada uno de estos procedimientos, estando cada una separada en su archivo propio: *memory_allocation.py* para albergar los algoritmos y sus procedimientos correspondientes, y *directions_translator.py* para la sección de traducción de direcciones.

Alineado con esto, es necesario ahondar en cómo están desarrollados ambos simuladores, cuya estructura aplica una POO robusta y que cumple con los requerimientos teóricos abordados en el curso. Para el primer caso, se disponen de 6 clases que se encargaran de “leer” los procesos que arriban a la memoria y gestionar su ubicación en ella. A continuación, sus respectivas explicaciones:

✓ **Clase ‘Process’:**

Su principal función es la de modelar el Bloque de Control de Procesos (PCB) que dispondrá cada entidad despachable involucrada, pero su principal diferencia con una definición orientada a tratar detalladamente estas entidades despachables, es que sólo almacenará la etiqueta única del proceso (PID) y el tamaño que éste ocupará, ya que el contexto requiere estrictamente de estos datos para funcionar. Además, al modelarse como objetos de comportamiento inmutable (ya que sus datos iniciales no serán editados, sino leídos), garantiza que el énfasis del simulador se centre en la manera en que serán alojados dentro de las particiones correspondientes y, con ello, poder ejecutar los cálculos de las métricas de manera más sencilla y directa.

✓ Clase 'MemoryBlock':

Como su nombre lo traduce, representa cada bloque de memoria (partición) física con el que se dispondrá en la RAM, entendiendo cuánto espacio es el que tiene, cuánto espacio ya ha sido ocupado, cuánto espacio le resta y qué procesos contiene. Es importante aclarar la última característica (o atributo), pues bajo el enfoque abordado por el curso, cada bloque es capaz de almacenar uno o más procesos en su interior, solamente dependiendo del espacio vacío que disponga, siendo posible simular ese comportamiento teórico gracias a la implementación de una estructura de datos de **lista (List)** del lenguaje de programación.

Dicha estructura es la ideal para este comportamiento, debido a que permite el acceso a todos los elementos (procesos) dentro de la partición, al igual que su eliminación de ella, sin necesidad de establecer un orden para hacerlo. Es decir, que dado que el bloque sólo conoce qué procesos están en su interior, cuando se desee liberarle su espacio nuevamente, podrá hacerlo de manera conmutativa, quitando a cada uno de su lugar y vaciando la lista sin problemas, y de manera análoga actuará cuando un proceso llega adentro de ella, anexándolo de forma directa y sin establecer un orden, lo cual agiliza computacionalmente su funcionamiento en el simulador.

Así mismo, esta clase permite determinar ciertas métricas de manera preliminar e individual, a través de métodos (establecidos como si fueran atributos, gracias al decorador de Python "*@property*") encargados de calcular y "almacenar" (de cierta manera) cuánto espacio están ocupando los procesos en su interior, la diferencia exacta entre la capacidad física del bloque y el espacio ocupado, el monitoreo constante del espacio libre en la partición (fragmentación interna) y si el bloque está totalmente vacío o no (*'used_space()'*, *'remaining_space()'*, *'internal_fragmentation()'* e *'is_free()'*, respectivamente), arrojando

valores que, sumados a todo el análisis de los bloques totales que se dispongan, determinarán las métricas solicitadas en el enunciado del examen. Cabe aclarar que la decisión de usar el decorador se basa en mejorar la legibilidad y comprensión del código, permitiendo acceder a los valores que generan estos métodos como si fueran atributos de un objeto del tipo MemoryBlock.

✓ Clase 'AllocationStrategy(ABC)':

Ahora bien, esta clase es una de las más importantes dentro de este primer simulador, puesto que maneja un comportamiento general para los algoritmos que se emplearán en él. Dicha clase es capaz de realizar esto, pues se plantea como una clase abstracta que determina el comportamiento base con el que contarán los procedimientos de asignación, es decir, actúa como la clase padre de los algoritmos a usar. Para ello, se utiliza el módulo 'abc' nativo del lenguaje, para formular una interfaz que adaptarán las demás clases suscritas a ésta, y su único y principal método (marcado con la etiqueta "*@abstractmethod*", pues no tiene una implementación estricta) '*find_block()*', funciona como el mecanismo que tendrá cada una de las clases hijas para determinar la partición más adecuada para alojar al proceso que estén evaluando, variando dicho bloque con las políticas de cada algoritmo, claro.

Complementando esta decisión de hacerlo así, ya que todos los algoritmos aquí usados deberán hacer ese proceso de búsqueda de forma general, se estipuló la aplicación del patrón de diseño Strategy, el cual postula diseñar una familia de componentes (en este caso, los algoritmos) y solamente variar la manera en cómo realizar una acción. Es decir, se crean dichas familias (clase padre abstracta y clases hijas) y, para evitar el uso excesivo de condicionales para evaluar qué es lo que se requiere hacer, simplemente se definen los comportamientos generales dentro de la interfaz y son las clases específicas las que se encargan de, valga la redundancia, especificar qué hacer en ese método, aplicando explícitamente el concepto de polimorfismo.

✓ **Clase 'FirstFit(AllocationStrategy)':**

Es la primera de las clases hijas mencionadas con anterioridad, la cual se encarga de implementar el algoritmo de Primer Ajuste (First Fit) dentro del simulador. Al aplicar el concepto de herencia mediante el patrón definido arriba, su funcionalidad es la de tomar el método *find_block()* y adaptar su funcionamiento al de buscar bloque por bloque en la RAM y evaluar si el tamaño que tienen es mayor o igual al del proceso sobre el que se está aplicando, y apenas halle alguno que cumpla con esta condición, le asignará el proceso en su interior, haciendo tangible la función de alojar un proceso en el primer bloque en el que encaje.

✓ **Clase 'BestFit(AllocationStrategy)':**

Similar al comportamiento de su “hermano”, esta clase hija adapta el método *find_block()* bajo la necesidad del algoritmo de Mejor Ajuste (Best Fit), donde se encarga de recorrer todas las particiones en RAM y, comparando una a una si su tamaño es quien menor diferencia tenga con el tamaño del proceso (es decir, que el resultado de la resta entre el tamaño ofrecido por el bloque menos el tamaño del proceso sea lo más pequeño posible), determina alojar al proceso en dicho bloque, una vez más haciendo plausible la aplicación de este procedimiento de encontrar el lugar más conveniente para una instancia activa.

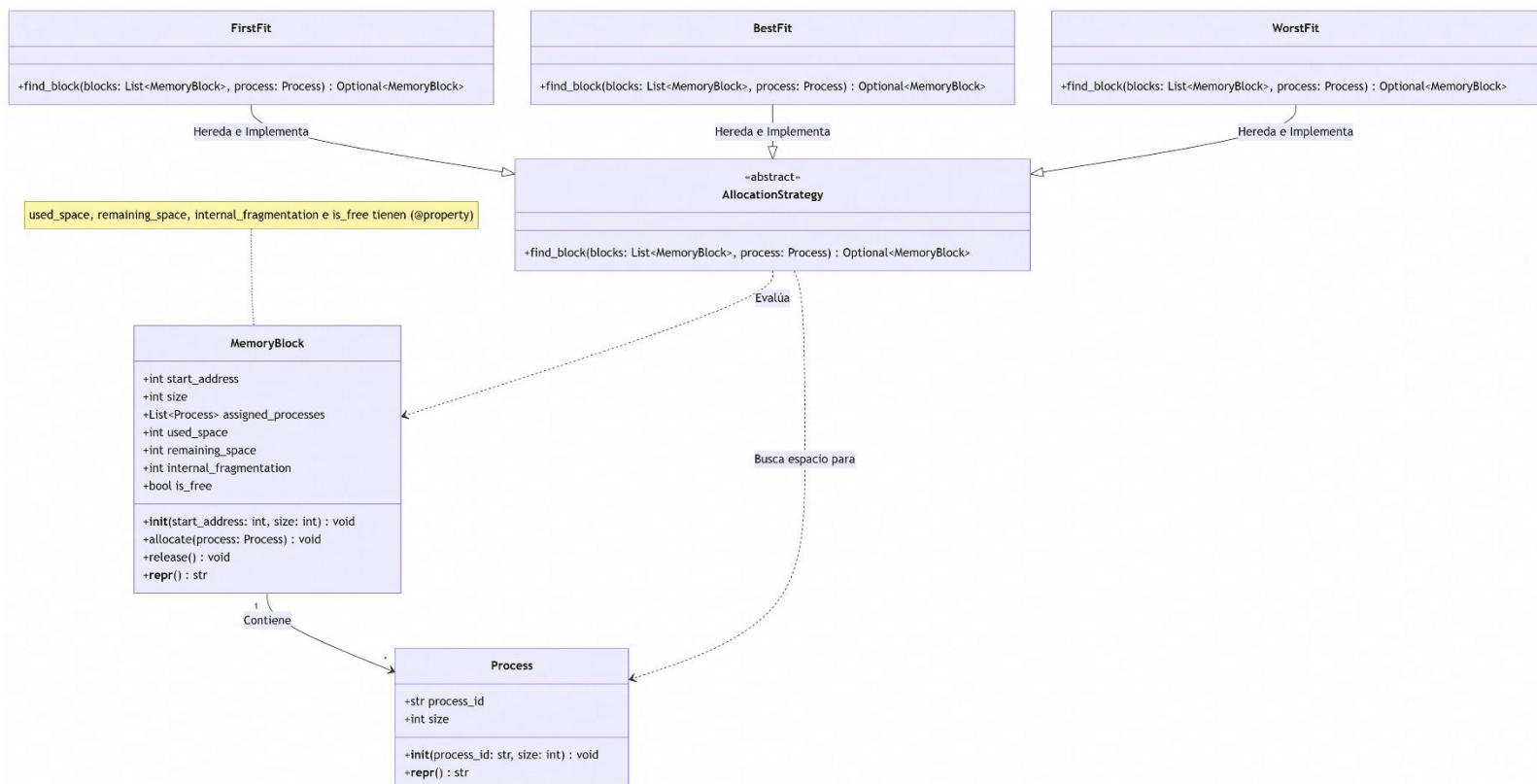
✓ **Clase 'WorstFit(AllocationStrategy)':**

Finalmente, la última de las clases hijas bajo este patrón, es la encargada de modificar el mismo método recientemente usado (*find_block()*), para llevar a cabo la tarea del algoritmo de Peor Ajuste (Worst Fit). Para ello, recorre todos los bloques que haya en la RAM, y evalúa contrariamente a la clase anterior: verifica que la diferencia entre el tamaño del bloque y el tamaño del proceso sea lo más grande posible, con el fin de poder alojar ahí un proceso y evitar la fragmentación interna lo máximo

posible, trayendo a la realidad computacional este tipo de soluciones para administrar la memoria principal.

De esta manera, es como se ha planteado el primer simulador de esta práctica, aclarando finalmente que los tres algoritmos descritos hacen el recorrido de las particiones a través de una lista como estructura de datos que reciben de parámetro, implementando así el puntero que indica en qué partición se encuentra y hacia cuál ir, lo cual es sencillo de modelar gracias a dicha estructura.

Así mismo, se dispone de un diagrama UML, específicamente un diagrama de clases, para comprender las relaciones que hay en el archivo y simulador.



2.2. Implementación de la Traducción de Direcciones Virtuales a Direcciones Físicas.

Paralelamente, para la implementación del segundo simulador encargado de traducir las direcciones virtuales dictadas a direcciones físicas en la memoria principal, se ha desarrollado un código basado en el esquema de **paginación de un nivel**, debido a que es el mecanismo que mayor relación directa posee con otro gran componente dentro de la administración de RAM: la Memoria Virtual (MV), y esta relación se basa en que la paginación permite efectuar la división, gestión y sobre todo traducción de los espacios de memoria física a bloques que se les asignan a los procesos, para dar la impresión de encapsulamiento a cada uno de ellos cuando en verdad se pueden hallar dispersos. Es decir, que la paginación de un nivel es el esquema elegido para esta implementación, debido a su gran simplicidad y velocidad para acceder, en este caso, a direcciones físicas.

Con ello en mente, se procede a describir cada una de las 4 clases usadas en el código mencionado (puesto que también se usó POO), para determinar el comportamiento adecuado para su funcionamiento.

✓ **Clase 'PageFrame':**

Es la encargada de representar cada uno de los Marcos de Página (Page Frames) en los que se divide de manera física y uniforme la memoria RAM real. Sus atributos base se centran en registrar su identificador de posición en el bloque de memoria y su capacidad fija en bytes dentro del mismo. Con el propósito de controlar su estado operativo, dispone de atributos que almacenan el proceso o la parte de uno que está alojado en ese marco, al igual que el número de página asociado a éste, emulando los registros de ocupación. De igual manera, mediante el uso del decorador “@property”, se implementó el método a funcionar como atributo ‘is_available()’, cuya función es la de dictaminar, de forma

booleana, si el marco está libre, es decir, que haya una ausencia de un proceso asignado. Y, como complemento, encapsula los métodos '*occupy()*' y '*release()*', los cuales restringen la alteración de los estados internos de los marcos, permitiendo observar de forma limpia los ciclos de concesión y liberación de celdas físicas de memoria.

✓ Clase '**PageTable**':

En este caso, se modeló una de las abstracciones lógicas más importantes del sistema, pues define la estructura de datos que el sistema operativo asocia a cada proceso, para registrar sus mapeos de memoria virtual, es decir, para lograr identificar qué marcos físicos son los que están albergando parte o todo el proceso en cuestión. El núcleo de esta clase, se basa en la utilización de la estructura de datos de **diccionario privado** del lenguaje, y se emplea puesto que es la más óptima para el propósito de realizar el mapeo descrito, debido a que permite indexar tuplas del estilo clave-valor de manera rápida para ejecutar operaciones de inserción y búsqueda de estos procesos a lo largo de la RAM. De esta forma, se logra establecer el comportamiento de una **tabla (map)** dentro del código, pudiendo actuar bajo la premisa de que las claves representan los números de página lógica y los valores corresponden a los marcos de página físicos asignados.

Así, para salvaguardar la integridad de estos mapeos frente a modificaciones externas que puedan corromper la simulación, el método '*current_mappings()*' expone una copia explícita del diccionario mientras que el método '*restore_mappings()*', logrando así una persistencia entre los datos que se tienen de un proceso específico y la ubicación real en RAM, traduciendo fidedignamente la o las locaciones del mismo.

✓ Clase '**Mmu**':

Su función es la de representar el componente de Unidad de Gestión de Memoria (MMU) dentro del simulador, siendo uno de los aspectos más importantes con los que se cuentan.

Basa su lógica en interceptar las peticiones de traducción de direcciones en tiempo de ejecución, tal como se describe en el curso, modelando el comportamiento del hardware al momento de recepcionar estas solicitudes del procesador. Bajo ese concepto, esta clase no almacena estados ni registros de procesos; funciona como un componente configurado con un tamaño de página inmutable (*'_page_size'*), y ejecuta solamente procedimientos aritméticos para “calcular” en qué dirección física se halla un proceso, logrando eso con ayuda de sus métodos *'descompose_virtual_address()'* (encargado de separar una dirección lógica mediante operadores de división entera (*'/'*) y módulo (*'%'*) para extraer, de forma exacta, el número de página virtual (*'p'*) y su respectivo desplazamiento u offset (*'d'*), tal como se abordó en clase) y *'translate()'* (que opera como un convertidor que recibe la dirección virtual y la instancia de PageTable asociada al proceso en ejecución, leyéndola y realizando la búsqueda del marco físico, donde si la referencia es exitosa, aplica la fórmula física para retornar la dirección física real; o, en caso contrario, detecta una referencia inválida e interrumpe el flujo, notificando formalmente la ocurrencia de un Fallo de Página (Page Fault). De este modo, es posible hacer la traducción de memoria virtual a memoria física, bajo los conceptos del curso.

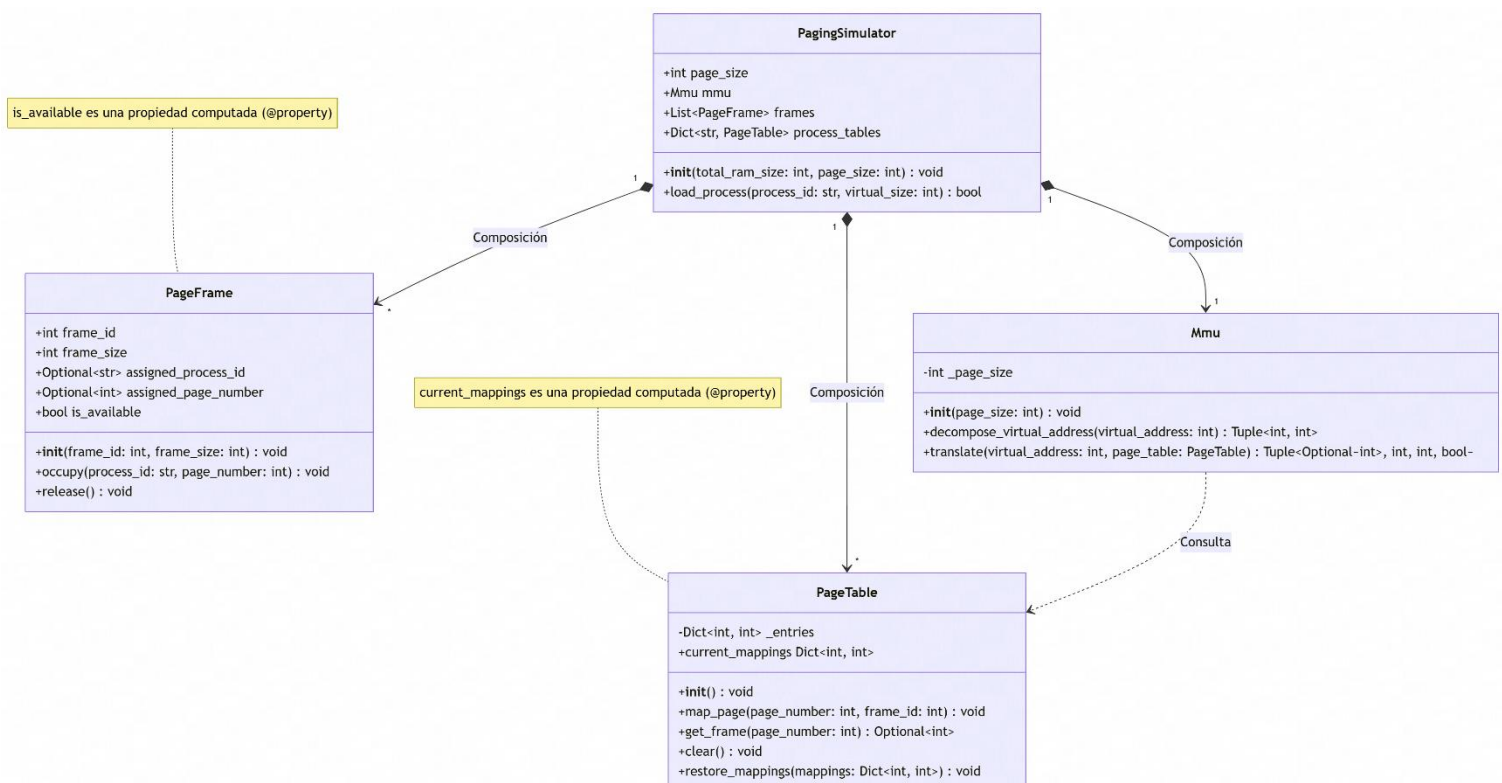
✓ Clase **'PagingSimulator'**:

Por último, pero no menos importante, se dispone de esta clase, cuyo comportamiento es el de ser la orquestadora principal de todo el funcionamiento del simulador de traducción. En esencia, su función principal es la de unificar el hardware físico (representado por una lista indexada de objetos PageFrame) con las capas lógicas del kernel (un diccionario global de tablas de páginas ingresadas y asociadas por proceso). Cuando se invoca, calcula dinámicamente el número de marcos totales basándose en la capacidad física total y el tamaño de página parametrizado. Para ello, su método *'load_process()'* implementa la lógica de asignación calculando el límite superior de páginas requeridas por el tamaño virtual del proceso mediante la fórmula: (*'virtual_size +*

`self.page_size - 1) // self.page_size`. Posteriormente, evalúa la disponibilidad en el pool físico y, haciendo uso del método `'pop(0)'` sobre la lista de marcos disponibles, se implementa la estructura de datos de **cola**, que tendrá un comportamiento FIFO, pudiendo así reservar secuencialmente los espacios necesarios para mapearlos de forma automatizada e inmediata en la tabla de páginas del proceso respectivo, garantizando que un proceso sólo se aloje si la memoria RAM cuenta con la capacidad total de marcos requeridos.

Es así como, luego de entender el funcionamiento y comportamiento de ambos simuladores, se logra ver cómo la POO y el correcto diseño de cada una de las clases implicadas en ambos casos, permite hacer realidad procesos complejos que involucren la “manipulación” de la memoria RAM en equipos de cómputo, mediante un sistema operativo.

De la misma manera, aquí está el diagrama correspondiente.



3. Pruebas para Validación del Funcionamiento.

Teniendo ya claros los comportamientos de los simuladores, es importante llevar a la realidad sus funcionamientos, siempre de forma correcta.

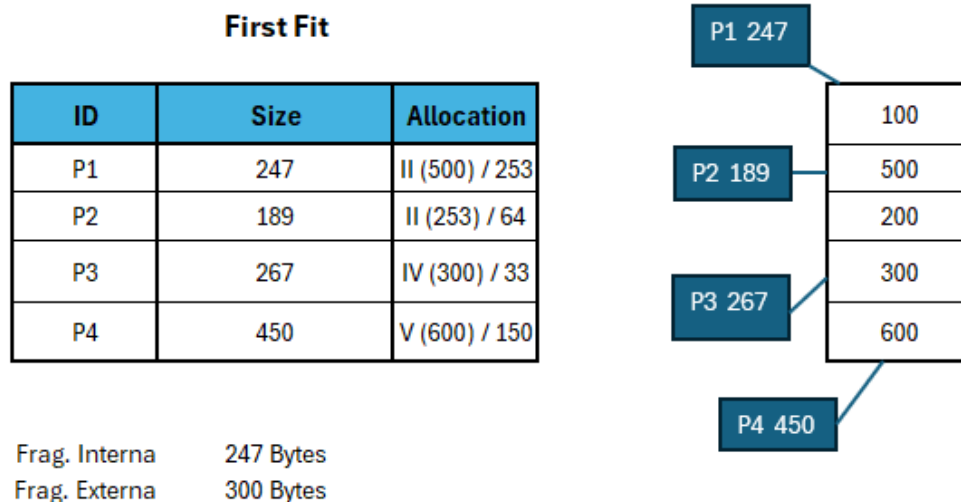
Por tal motivo, a continuación se muestran las pruebas desarrolladas para cada uno de ellos, buscando comparar que los cálculos obtenidos de forma “manual”, aplicando los conceptos y procedimientos vistos en clase, sea exactamente el mismo bajo la ejecución de los dos programas diseñados, demostrando de manera verosímil su correctitud.

3.1. Casos de Prueba para la Primera Implementación.

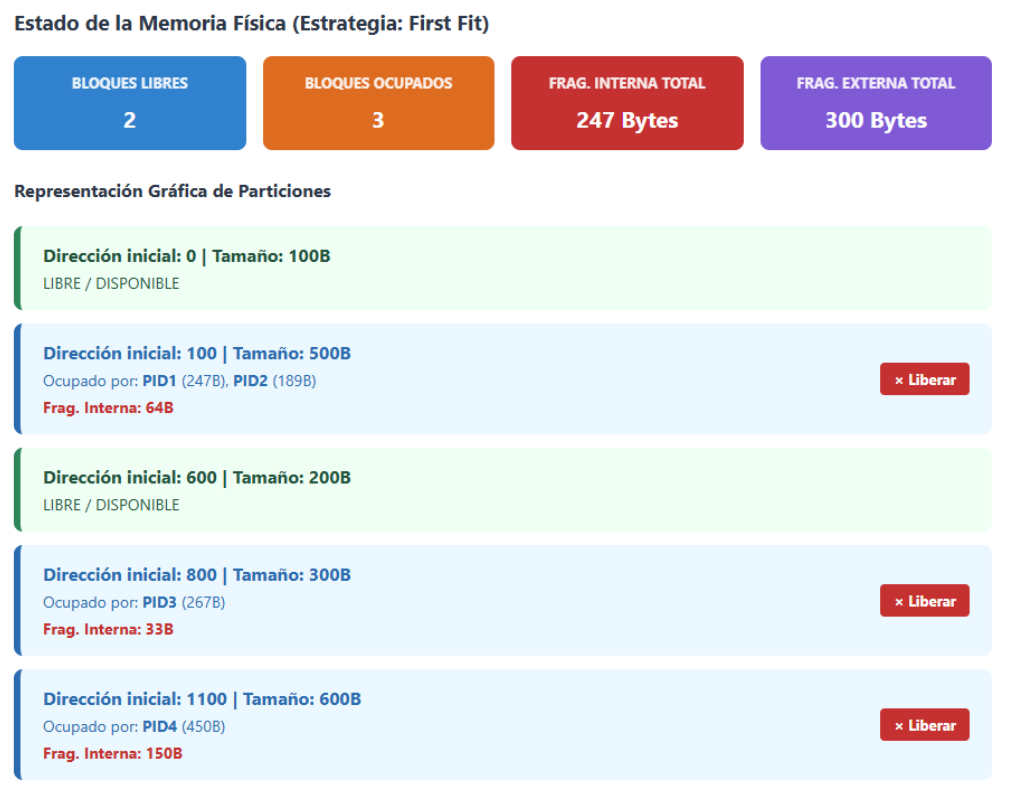
Para la primera implementación, cuyo objetivo es el de asignar de forma correcta la partición o bloque sobre el que reposará cada proceso en RAM, se desarrollaron 3 ejercicios que ponen a prueba la distribución que maneja cada algoritmo empleado, además de generar un cuarto enunciado que permita ver el actuar de cada uno de los algoritmos bajo la misma entrada, con el fin de observar, analizar y concluir qué procedimiento de asignación es el más adecuado en ese contexto.

- Primera prueba (aplicación del algoritmo First Fit).

De manera teórica:

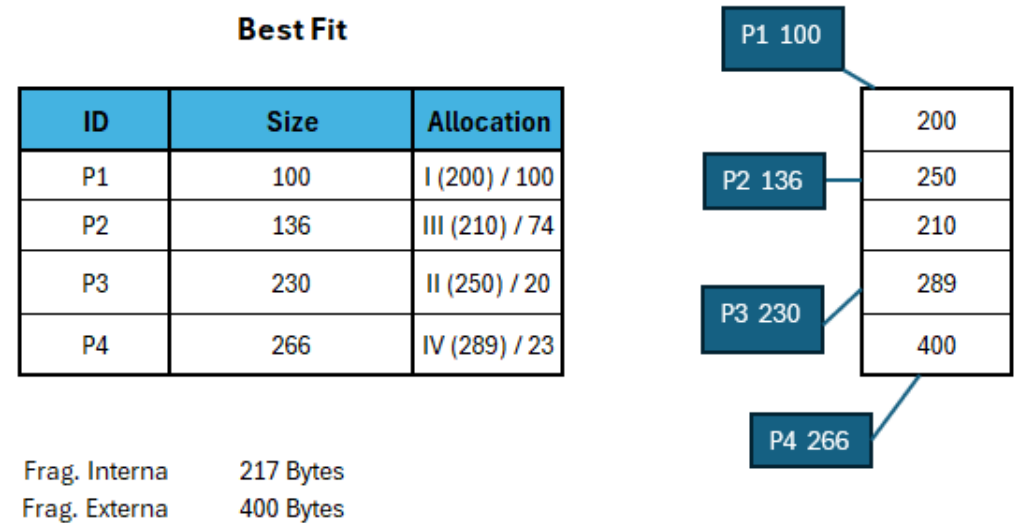


Arrojado por el sistema:

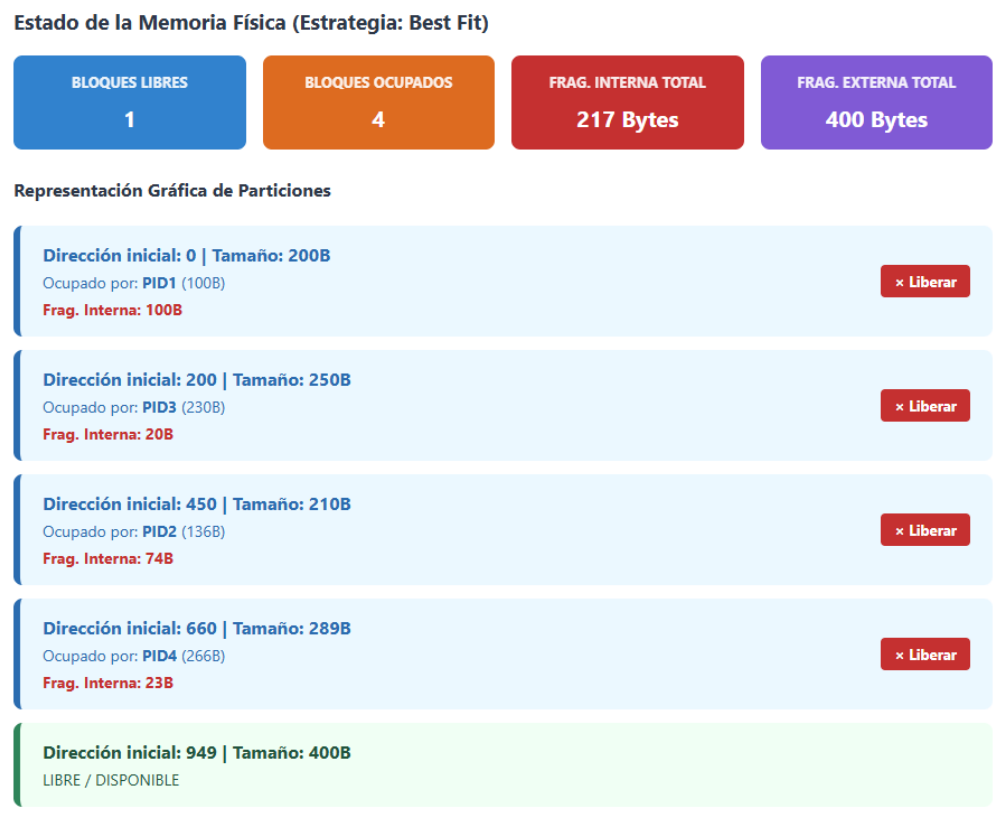


- Segunda prueba (aplicando el algoritmo Best Fit).

De manera teórica:

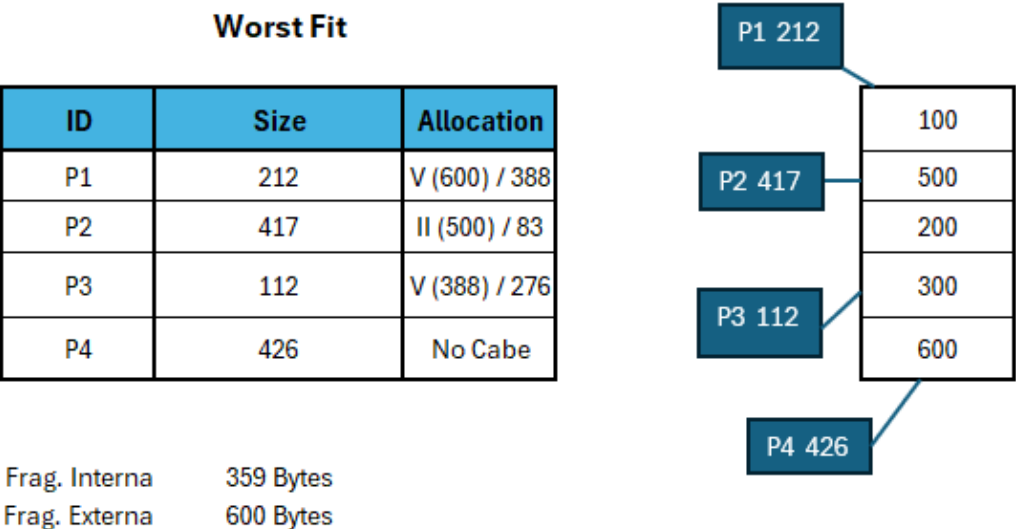


Arrojado por el sistema:

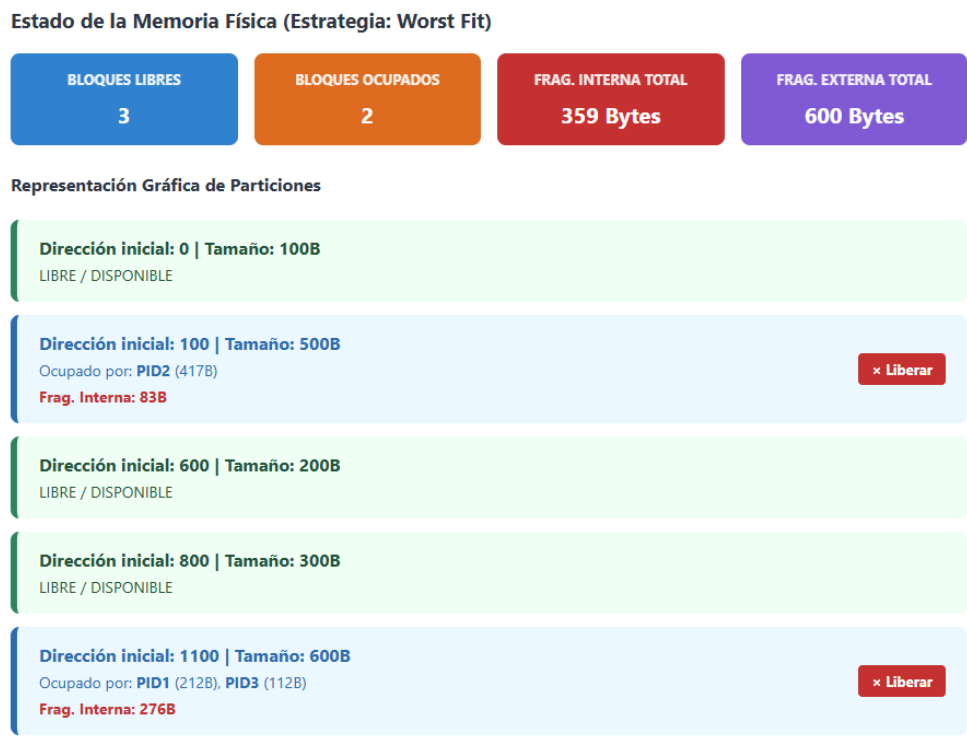


- Tercera prueba (aplicando el algoritmo Worst Fit).

De manera teórica:

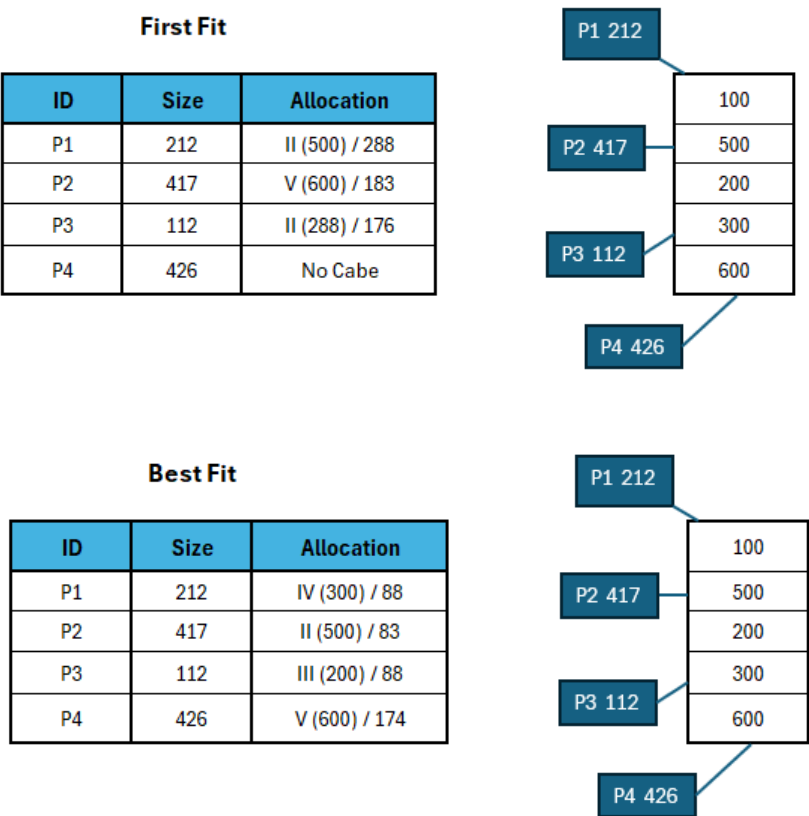


Arrojado por el sistema:



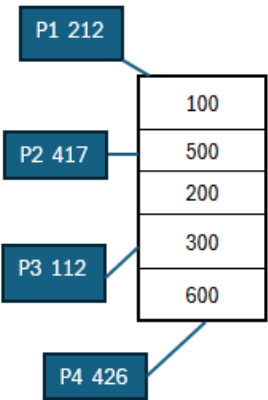
- Cuarta prueba (aplicando todos los algoritmos para una misma entrada).

De manera teórica:



Worst Fit

ID	Size	Allocation
P1	212	V (600) / 388
P2	417	II (500) / 83
P3	112	V (388) / 276
P4	426	No Cabe



Arrojado por el sistema:

Estado de la Memoria Física (Estrategia: First Fit)

BLOQUES LIBRES	BLOQUES OCUPADOS	FRAG. INTERNA TOTAL	FRAG. EXTERNA TOTAL
3	2	359 Bytes	600 Bytes

Representación Gráfica de Particiones

Dirección inicial: 0 Tamaño: 100B LIBRE / DISPONIBLE
Dirección inicial: 100 Tamaño: 500B Ocupado por: PID1 (212B), PID3 (112B) Frag. Interna: 176B × Liberar
Dirección inicial: 600 Tamaño: 200B LIBRE / DISPONIBLE
Dirección inicial: 800 Tamaño: 300B LIBRE / DISPONIBLE
Dirección inicial: 1100 Tamaño: 600B Ocupado por: PID2 (417B) Frag. Interna: 183B × Liberar

Estado de la Memoria Física (Estrategia: Best Fit)

BLOQUES LIBRES	BLOQUES OCUPADOS	FRAG. INTERNA TOTAL	FRAG. EXTERNA TOTAL
1	4	433 Bytes	100 Bytes

Representación Gráfica de Particiones

Dirección inicial: 0 Tamaño: 100B LIBRE / DISPONIBLE	
Dirección inicial: 100 Tamaño: 500B Ocupado por: PID2 (417B) Frag. Interna: 83B	× Liberar
Dirección inicial: 600 Tamaño: 200B Ocupado por: PID3 (112B) Frag. Interna: 88B	× Liberar
Dirección inicial: 800 Tamaño: 300B Ocupado por: PID1 (212B) Frag. Interna: 88B	× Liberar
Dirección inicial: 1100 Tamaño: 600B Ocupado por: PID4 (426B) Frag. Interna: 174B	× Liberar

Estado de la Memoria Física (Estrategia: Worst Fit)

BLOQUES LIBRES	BLOQUES OCUPADOS	FRAG. INTERNA TOTAL	FRAG. EXTERNA TOTAL
3	2	359 Bytes	600 Bytes

Representación Gráfica de Particiones

Dirección inicial: 0 Tamaño: 100B LIBRE / DISPONIBLE	
Dirección inicial: 100 Tamaño: 500B Ocupado por: PID2 (417B) Frag. Interna: 83B	× Liberar
Dirección inicial: 600 Tamaño: 200B LIBRE / DISPONIBLE	
Dirección inicial: 800 Tamaño: 300B LIBRE / DISPONIBLE	
Dirección inicial: 1100 Tamaño: 600B Ocupado por: PID1 (212B), PID3 (112B) Frag. Interna: 276B	× Liberar

Ahora, teniendo ya disponibles las tablas donde se evaluó con cada algoritmo de gestión, se procede a formular una tabla que encapsula las métricas obtenidas.

Métrica	First Fit	Best Fit	Worst Fit
Procesos Asignados	3 / 4 (P1, P2 y P3)	4 / 4 (Todos)	3 / 4 (P1, P2 y P3)
Procesos Bloqueados	1 (P4)	0	1 (P4)
RAM Utilizada	741 Bytes	1167 Bytes	741 Bytes
Fragmentación Interna	359 Bytes	433 Bytes	359 Bytes
Fragmentación Externa	600 Bytes	100 Bytes	600 Bytes

Para este set de datos específico, **Best Fit** demostró ser el algoritmo óptimo. Al emparejar con precisión quirúrgica las demandas con las capacidades, minimizó la fragmentación interna a 433 Bytes y la externa a sólo 100 Bytes, permitiendo la carga del 100% de los procesos. Por el contrario, *First Fit* y *Worst Fit* cayeron en malas decisiones tempranas, fragmentando la memoria y dejando al sistema incapaz de soportar el tamaño de P4, generando rechazo de carga.

De esta manera, contrastando con la teoría aplicada, se puede observar el buen comportamiento del simulador, soportando todos los algoritmos usados.

3.2. Casos de Prueba para la Segunda Implementación.

Por su parte, la segunda implementación, cuyo objetivo es el de traducir las direcciones virtuales pasadas a direcciones físicas en la memoria, también será puesta a prueba bajo 3 ejercicios que ponen a prueba la lógica implementada. Así, primero se presentará el desarrollo de cada prueba de forma teórica, para después enseñar lo arrojado por el sistema y comprobar que se obtengan exactamente los mismos resultados en ambos casos. Se usó un proceso cuyo tamaño es de 12288 Bytes y otro cuyo tamaño es de 4096 Bytes.

- Primera prueba (traducir dirección virtual 6000).

De manera teórica:

(Nota: Para representarlo, utilicé el lenguaje LaTeX).

Caso 1: Dirección Virtual (V_a) = 6000 (Sin procesos previos)

1. **Descomposición de la Dirección (Aritmética de la MMU):** Número de Página (p) = $6000 // 4096 = 1$
 $Desplazamiento(d) = 6000 \% 4096 = 1904$
2. **Consulta de la Tabla de Páginas:**
Al estar la memoria vacía, la página lógica $p = 1$ fue asignada por el simulador de forma secuencial en el **Marco 1** ($frame_id = 1$).
3. **Cálculo de la Dirección Física Real (P_a):** $P_a = (frame_id \times S_p) + d$
 $P_a = (1 \times 4096) + 1904$
 $P_a = 4096 + 1904 = 6000$
4. **Resultado:** Traducción Exitosa ($P_a = 6000$).

Arrojado por el sistema:

Resultado de la Unidad de Gestión de Memoria (MMU)

Proceso: **PID_A** | Dirección Virtual Consultada: **6000**

Paso 1 (Descomposición Matemática): Dirección 6000 / Página 4096 = **Página Virtual #1** | Residuo = **Desplazamiento (Offset) 1904**

Paso 2 (Mapeo de Tabla): La Página Virtual 1 está cargada en el **Marco Físico #1**.

Dirección Física Real Calculada: 6000

Pool de Marcos de la Memoria Física (RAM Real)

Marco 0

Proceso: **PID_A**

Pág. Lógica: #0

Marco 1

Proceso: **PID_A**

Pág. Lógica: #1

Marco 2

Proceso: **PID_A**

Pág. Lógica: #2

Marco 3

DISPONIBLE

Marco 4

DISPONIBLE

Marco 5

DISPONIBLE

Marco 6

DISPONIBLE

Marco 7

DISPONIBLE

Mapeo de Tablas de Páginas Activas

Tabla de Páginas: PID_A

Página Lógica	Marco Físico (RAM)
Página Virtual 0	Marco 0
Página Virtual 1	Marco 1
Página Virtual 2	Marco 2

- Segunda prueba (traducir 13000).

De manera teórica:

Caso 2: Dirección Virtual (V_a) = 13000 (Demostración de Page Fault)

- Descomposición de la Dirección (Aritmética de la MMU):** Número de Página (p) = $13000 // 4096 = 3$
 $Desplazamiento(d) = 13000 \% 4096 = 712$
- Consulta de la Tabla de Páginas:**
Al intentar acceder al índice $p = 3$ dentro del diccionario de la tabla de páginas del proceso P1, la MMU determina que dicha clave **no existe**, dado que el proceso únicamente tiene asignadas y mapeadas las páginas lógicas 0, 1 y 2 (12288B en total).
- Mecanismo de Excepción de Hardware:**
Al no encontrarse una referencia física válida para la página solicitada, el simulador interrumpe el flujo de traducción normal y genera de manera controlada una excepción de **Fallo de Página (Page Fault)**.
- Resultado en la Interfaz:** El sistema captura la excepción y despliega una alerta Flash informativa en la GUI notificando el direccionamiento inválido.

Arrojado por el sistema:

Resultado de la Unidad de Gestión de Memoria (MMU)

Proceso: **PID_A** | Dirección Virtual Consultada: **13000**

Paso 1 (Descomposición Matemática): Dirección 13000 / Página 4096 = **Página Virtual #3** | Residuo = **Desplazamiento (Offset) 712**

PAGE FAULT (Fallo de Página): La Página Virtual 3 no se encuentra mapeada en la memoria RAM física actual.

Pool de Marcos de la Memoria Física (RAM Real)

Marco 0
Proceso: **PID_A**
Pág. Lógica: #0

Marco 1
Proceso: **PID_A**
Pág. Lógica: #1

Marco 2
Proceso: **PID_A**
Pág. Lógica: #2

Marco 3
DISPONIBLE

Marco 4
DISPONIBLE

Marco 5
DISPONIBLE

Marco 6
DISPONIBLE

Marco 7
DISPONIBLE

Mapeo de Tablas de Páginas Activas

Tabla de Páginas: PID_A

Página Lógica	Marco Físico (RAM)
Página Virtual 0	Marco 0
Página Virtual 1	Marco 1
Página Virtual 2	Marco 2

- Tercera prueba (traducir 6000 con otro proceso alojado).

De manera teórica:

Caso 3: Dirección Virtual (V_a) = 6000 (Con Proceso Fantasma Ocupando Marco 0)

- Efecto en el Entorno Lógico:**
Al cargar previamente un proceso P_{ocupa} de 4096B, este toma de forma automática el **Marco 0**. Por ende, al alojar el proceso principal **P1**, sus páginas lógicas se desplazan secuencialmente en el pool físico de la RAM: la página 0 va al Marco 1, la **página 1 al Marco 2** y la página 2 al Marco 3.
- Descomposición de la Dirección ($V_a = 6000$):** Número de Página (p) = $6000 // 4096 = 1$
 $Desplazamiento(d) = 6000 \% 4096 = 1904$
- Consulta de la Tabla de Páginas:**
La MMU lee la tabla indexada del proceso **P1** y extrae que para la clave de página $p = 1$, el mapeo físico apunta de forma dinámica al **Marco 2** ($frame_id = 2$).
- Cálculo de la Dirección Física Real (P_a):** $P_a = (frame_id \times S_p) + d$
 $P_a = (2 \times 4096) + 1904$
 $P_a = 8192 + 1904 = 10096$
- Resultado:** Traducción Dinámica Exitosa ($P_a = 10096$).

Arrojado por el sistema:

Subsistema de Memoria por Paginación

Tamaño de Página de la Cátedra: 4096 Bytes

Resultado de la Unidad de Gestión de Memoria (MMU)

Proceso: **PID_A** | Dirección Virtual Consultada: **6000**

Paso 1 (Descomposición Matemática): Dirección 6000 / Página 4096 = **Página Virtual #1** | Residuo = **Desplazamiento (Offset) 1904**

Paso 2 (Mapeo de Tabla): La Página Virtual 1 está cargada en el **Marco Físico #2**.

Dirección Física Real Calculada: 10096

Pool de Marcos de la Memoria Física (RAM Real)

Marco 0

Proceso: **P_ocupa**

Pág. Lógica: #0

Marco 1

Proceso: **PID_A**

Pág. Lógica: #0

Marco 2

Proceso: **PID_A**

Pág. Lógica: #1

Marco 3

Proceso: **PID_A**

Pág. Lógica: #2

Marco 4

DISPONIBLE

Marco 5

DISPONIBLE

Marco 6

DISPONIBLE

Marco 7

DISPONIBLE

Mapeo de Tablas de Páginas Activas

Tabla de Páginas: PID_A

Página Lógica	Marco Físico (RAM)
Página Virtual 0	Marco 1
Página Virtual 1	Marco 2
Página Virtual 2	Marco 3

Tabla de Páginas: P_ocupa

Página Lógica	Marco Físico (RAM)
Página Virtual 0	Marco 0

Tal como se ha comprobado visual y teóricamente, ambos procedimientos (teórico y algorítmico) arrojan los mismos valores exactos para cada uno de los casos manejados, además de mostrar en la GUI los demás aspectos requeridos por el enunciado del parcial, demostrando la correcta implementación de cada simulador aquí abordado.

4. Uso Responsable y Transparente de la IA.

Dando cumplimiento estricto a los lineamientos de originalidad y transparencia solicitados para esta entrega, y abordando esta sección de manera más personal y honesta, presento los registros del uso que realicé de Inteligencia Artificial como herramienta de apoyo técnico para la optimización de la calidad del software, la comprensión de conceptos avanzados de hardware y la configuración del entorno de Docker en mi máquina local. De esta manera, detallaré los prompts que empleé y la respectiva personalización humana aplicada para cada uno.

Prompt 1: Comprensión Teórica y Modelado de la MMU.

Prompt suministrado a la IA: *"Hola Gemini, buenas tardes. Estoy desarrollando un examen parcial de Sistemas Operativos y necesito de tu ayuda. Debo construir un simulador en Python que emule la traducción de direcciones en memoria virtual por paginación de un nivel. Ya entiendo la teoría de cómo la MMU descompone la dirección lógica usando división entera y residuo, pero me cuesta visualizar cómo modelar la MMU en código POO sin que almacene estados de los procesos. ¿Me podrías dar una guía teórica de cómo estructurar una clase abstracta o pura que reciba la dirección virtual y la tabla de páginas del proceso activo para calcular la dirección física real y detectar fallos de página, por favor?"*

Personalización y ajuste propio realizado: A partir de la guía teórica proporcionada por la IA, basada en los conceptos vistos en clase, procedí a codificar la clase Mmu de manera independiente en su archivo correspondiente. Adapté la lógica para asegurar que el método de traducción retornara una tupla de cuatro valores (*'physical_address'*, *'page_num'*, *'offset'*, *'success'*), lo cual me facilitó la posterior integración con el backend de Flask, un framework de Python que utilicé en este desarrollo. Así mismo, pulí las descripciones de los fallos de página para que se alinearan de forma exacta con la terminología del curso, asegurando que la interrupción por hardware simulada fuera fiel a lo visto en las clases.

Prompt 2: Optimización de Arquitectura y Optimización de Enlace Backend-Frontend con Flask.

Prompt suministrado a la IA: *"Tengo un backend en Flask que orquesta dos simuladores de memoria independientes: uno contiguo y otro de paginación. El problema es que al ser una aplicación web, los estados de los objetos de mis clases (como MemoryBlock y PagingSimulator) se borran entre cada petición POST de los formularios de la GUI. Sé que puedo usar la cookie de sesión (session) de Flask, pero mis objetos no son serializables a JSON de forma nativa. ¿Qué funciones auxiliares de serialización y deserialización manual puedo construir para guardar y reconstruir fielmente mis estructuras de objetos y diccionarios privados en cada recarga sin corromper los datos?"*.

Personalización y ajuste propio realizado: Tomé la propuesta de serialización basada en diccionarios sugerida por la IA y la reestructuré minuciosamente para adaptarla a mis clases específicas. En particular, modifiqué profundamente la deserialización del simulador contiguo para soportar una lista dinámica de procesos cohabitando en un mismo bloque, asegurando que al reconstruir el estado no se perdiera la propiedad "@property" que calcula la fragmentación interna. También aseguré el tipado estricto convirtiendo explícitamente las claves del diccionario de mapeos de paginación a enteros (int(k)), corrigiendo un error de tipos que afectaba la visualización en la GUI.

Adjunto la evidencia:

```
class MemoryBlock: 19+ usages new *
    """Representa una partición o bloque contiguo de la memoria física."""

    def __init__(self, start_address: int, size: int) -> None: new *
        """Inicializa una partición de memoria orientada a asignación contigua.

        Args:
            start_address (int): Dirección física inicial del bloque.
            size (int): Capacidad total del bloque de memoria en bytes.
        """

        self.start_address: int = start_address
        self.size: int = size
        # Adaptación: Lista para cohabitación de múltiples procesos en la partición
        self.assigned_processes: List[Process] = []
```

```

@property 3+ usages new *
def internal_fragmentation(self) -> int:
    """Determina la fragmentación interna acumulada en el bloque.

    Returns:
        int: El espacio sobrante si el bloque está ocupado, o 0 si está libre.
    """
    if self.is_free:
        return 0
    return self.remaining_space

```

Prompt 3: Diseño de Microcopy y Mejora Visual de GUI.

Prompt suministrado a la IA: *"Estoy diseñando el archivo index.html para mi simulador de memoria y quiero que la interfaz tenga una distribución profesional en dos columnas utilizando flexbox o Grid CSS. Al lado izquierdo irán los formularios de configuración de hardware y al lado derecho la visualización visual de la RAM. Necesito que me ayudes a estructurar las clases de CSS para que el pool de marcos de página en la paginación virtual se organice como una cuadrícula limpia y que la representación de los bloques contiguos cambie de color dinámicamente si están LIBRES u OCUPADOS. Además, sugiéreme textos de microcopy claros para indicarle al usuario que debe inicializar el hardware antes de meter procesos".*

Personalización y ajuste propio realizado: Si bien la IA me proporcionó las reglas base de CSS, realicé una personalización estética profunda para que el diseño visual quedara milimétricamente alineado y balanceada. Modifiqué los colores de las tarjetas de métricas por tonos pasteles más legibles y diseñé un contenedor de mensajes Flash personalizado para las alertas de error de la UX. En cuanto al microcopy, reescribí por completo los mensajes sugeridos por la IA para darles un tono más académico y formal, como el banner informativo que bloquea el formulario de procesos contiguos si la memoria no ha sido inicializada previamente.

(Nota: la evidencia puede apreciarse mucho mejor en el video de la sustentación, anexo al final del documento).

Prompt 4: Despliegue en Contenedor con Docker.

Prompt suministrado a la IA: *"Para la entrega de mi parcial de Sistemas Operativos, se exige la entrega del software para desplegarse con Docker. Estoy trabajando utilizando el IDE IntelliJ IDEA Ultimate. Mi proyecto tiene la estructura típica de Flask con un archivo application.py en la raíz y las clases lógicas en una carpeta src. ¿Cómo debe ser el archivo Dockerfile para compilar esta aplicación con Python 3.11, exponer el puerto 5000 de forma segura y asegurar que el contenedor se ejecute correctamente en cualquier entorno de evaluación?"*.

Personalización y ajuste propio realizado: Implementé el Dockerfile generado por la IA en mi entorno local, configurando los comandos en la raíz de mi espacio de trabajo. Durante el proceso de validación en mi terminal, identifiqué que era necesario mapear el host a la dirección 0.0.0.0 dentro del bloque 'if __name__ == '__main__':' de mi archivo controlador para que el puerto expuesto por el contenedor fuera accesible desde el navegador. Esta personalización garantizó la portabilidad absoluta del simulador y eliminó cualquier advertencia de red en mi IDE de desarrollo.

Adjunto la evidencia:

```
if __name__ == '__main__':  
    # Lanzar la aplicación en red abierta para correcta exposición del puerto en el contenedor Docker  
    app.run(host='0.0.0.0', port=5000, debug=True)
```

```
# Base liviana y segura de Python basada en Linux Alpine  
FROM python:3.11-alpine  
  
# Establecer el directorio de trabajo dentro del contenedor  
WORKDIR /app  
  
# Copiar primero el manifiesto de dependencias para aprovechar la caché de Docker  
COPY requirements.txt .  
  
# Instalar las dependencias de Flask sin almacenar basura temporal de instalación  
RUN pip install --no-cache-dir -r requirements.txt  
  
# Copiar todo el resto del código del proyecto al contenedor  
COPY . .  
  
# Informar el puerto en el que escuchará nuestra aplicación web  
EXPOSE 5000  
  
# Comando por defecto para arrancar el backend controlador cuando el contenedor inicie  
CMD ["python", "application.py"]
```

5. Conclusiones.

A partir del desarrollo de este componente práctico, se evidencia con claridad la pertinencia y potencia del uso de herramientas como el hardware simulado de la Unidad de Gestión de Memoria (MMU) y el esquema de traducción por tabla de páginas, las cuales demostraron ser un medio altamente expresivo y preciso para la emulación, control y direccionamiento de espacios lógicos disociados de la RAM física, evitando por completo el uso de variables globales desincronizadas, solapamientos en los bloques de memoria o instrucciones de traducción de direcciones ajenas a la lógica de la paginación y las políticas de ubicación observadas en el curso. El motor de control en los simuladores operó como un gestor natural de particionamiento y asignación, favoreciendo construcciones ligadas directamente al dominio de la administración de memoria real, especialmente en la captura inmediata de excepciones de hardware mediante fallos de página (Page Faults) y la evaluación de huecos residuales, proporcionando además una exactitud en el cálculo y devolución de las métricas de fragmentación interna y externa solicitadas, tal como se comprobó en la sección de pruebas, otorgando también una buena legibilidad y fiabilidad sobre la implementación.

En ese mismo orden, las colecciones de carácter estructurado bajo diccionarios y listas dinámicas, junto con la formación de tablas, poseen también una capacidad muy potente al momento de agrupar, indexar y despachar celdas de memoria, puesto que permiten organizarlas de forma secuencial y, debido a las restricciones de su implementación, actúan fielmente bajo las políticas FIFO (como en la extracción secuencial de marcos libres mediante '*pop(0)*') o mediante mapeos en la tabla de páginas, adaptándose de forma pura a las necesidades de cada estrategia interna, como First Fit, Best Fit o Worst Fit. Con tal característica, se origina un acceso a los datos muy directo y controlado, lo cual reduce la complejidad algorítmica y los recursos de hardware empleados para los procesos que impliquen transiciones y mapeos en el espacio direccionable, lo cual refuerza la recomendación y uso de este tipo de abstracciones

encapsuladas para todo software de simulación, en especial bajo las directrices de diseño y las reglas de fragmentación tratadas en el curso.

Y, globalmente, la experiencia de implementación refuerza la idea de que el diseño basado en el paradigma Orientado a Objetos no sólo es conceptualmente más limpio, sino también más robusto, seguro y escalable a nivel lógico. La posibilidad de razonar formalmente sobre el comportamiento de las entidades del sistema operativo (como el Bloque de Control de Procesos adaptado en Process, la partición física en MemoryBlock, y las estructuras físicas de los marcos de página), junto con la orquestación transparente de las políticas de asignación y traducción mediante el patrón Strategy, valida su idoneidad para tareas que requieren precisión matemática y estructura rigurosa, como la determinación de métricas globales e indicadores de desperdicio desde una perspectiva puramente modular, y gracias al uso de clases y abstracciones bien definidas, se pueden lograr modelar dichos comportamientos para resolver problemas complejos de la cotidianidad informática de forma más efectiva.

6. Enlaces de Entrega.

Finalmente, en esta sección se disponen de los enlaces correspondientes a cada elemento a evaluar.

Primero, el enlace al repositorio público de GitHub donde reposa el código fuente y los archivos de pruebas utilizados en este desarrollo:

<https://github.com/davidtaborda2025/Desarrollo-Parcial-2---Sistemas-Operativos.git>

Paralelamente, aquí también se añade el enlace al video de la sustentación del desarrollo aquí explicado, ubicado en YouTube:

https://youtu.be/0XQpgjIZAEY?si=l3oxStUiNdN_7PWA